PROYECTO INTEGRADOR — UNIDAD 2

Generación de código intermedio | Lenguajes y Autómatas II

Nombre del proyecto	MiniComp: generador de código intermedio
Modalidad	Trabajo en equipo con responsabilidades individuales
Producto final	Programa funcional en Python, pruebas y exposición breve

1. Propósito

Desarrollar en Python un programa educativo que reciba instrucciones de un lenguaje sencillo y genere distintas representaciones de código intermedio. El equipo simulará una parte del trabajo de un compilador: reconocer expresiones y asignaciones, identificar variables y constantes, y traducir las operaciones a notación prefija, infija y postfija, además de producir triplos y cuádruplos. También incorporará una representación básica de instrucciones de control y llamadas a funciones.

2. Situación que resolverá el programa

Se necesita una herramienta para que un estudiante escriba instrucciones de un pseudolenguaje y observe cómo podrían representarse antes de convertirse en código máquina. No tiene que ser un compilador profesional ni ejecutar todas las instrucciones: debe analizar un subconjunto definido, explicar errores básicos y mostrar claramente el código intermedio generado.

3. Lenguaje de entrada permitido

El programa debe aceptar, como mínimo, estas formas de instrucción:

```
entero a = 8;
entero b = 3;
entero resultado;
resultado = a + b * 2;
si (resultado > 10) {
resultado = resultado - 1;
}
imprimir(resultado);
```

Para mantener el alcance manejable, se permite trabajar con tipos entero y decimal; identificadores; números; operadores +, -, *, / y comparadores <, >, ==, !=; asignaciones; la estructura si; y llamadas a imprimir() y a una función definida por el equipo. El equipo debe documentar qué construcciones sí soporta.

4. Funcionalidades obligatorias

A. Lectura y validación. Leer instrucciones por teclado o desde un archivo .txt. Detectar errores sencillos: asignación sin expresión, paréntesis desbalanceados, identificador no declarado o símbolo no admitido.

B. Variables y constantes. Identificar nombres, valores y tipos básicos. Mantener una tabla de símbolos con identificador, tipo y valor inicial si existe.

C. Notaciones. Para cada expresión aritmética, mostrar la expresión infija original, su forma prefija y su forma postfija. Respetar precedencia y paréntesis.

D. Código intermedio. Generar triplos y cuádruplos para operaciones y asignaciones. Cada operación debe aparecer en un orden que permita reconstruir el cálculo.

E. Instrucciones de control. Traducir la estructura si a etiquetas y saltos simbólicos, por ejemplo IF_FALSE y GOTO. No es obligatorio ejecutar los saltos; sí mostrar su representación y destino.

F. Funciones. Reconocer al menos una llamada a función, como imprimir(resultado), y representarla mediante una instrucción intermedia. Si el equipo implementa una función propia, mostrar sus parámetros y llamada.

G. Presentación. Mostrar los resultados por secciones, con títulos claros, y permitir procesar más de un ejemplo sin reiniciar el programa.

5. Ejemplo de resultado esperado

Entrada:

```
resultado = a + b * 2;
```

El programa debe producir resultados equivalentes a los siguientes (los nombres temporales pueden variar):

```
Infija: a + b * 2
Prefija: + a * b 2
Postfija: a b 2 * +

Cuádruplos:
(1) (*, b, 2, t1)
(2) (+, a, t1, t2)
(3) (=, t2, -, resultado)

Triplos:
(1) (*, b, 2)
(2) (+, a, (1))
(3) (=, (2), resultado)
```

Nota: los triplos suelen referenciar el resultado de una operación por su número de instrucción. En los cuádruplos se usa un campo de resultado, por lo que pueden aparecer temporales como t1 y t2.

6. Organización sugerida del trabajo

El equipo puede distribuir las responsabilidades según el número de integrantes. Todas las personas deben comprender el flujo completo del programa y poder explicar al menos una parte del código.

Responsabilidad	Trabajo principal
1. Analizador de entrada	Leer instrucciones, separar tokens y validar errores básicos.
2. Expresiones y notaciones	Resolver precedencia y convertir infija a prefija/postfija.
3. Código intermedio	Generar triplos y cuádruplos a partir de las operaciones.
4. Control y funciones	Representar condicionales, etiquetas, saltos y llamadas a función.
5. Integración y pruebas	Unir módulos, preparar ejemplos, documentar y verificar resultados.

Si el equipo tiene menos integrantes, una persona puede asumir más de una responsabilidad. Si tiene más, pueden dividir las pruebas, la tabla de símbolos, la documentación o la interfaz.

7. Etapas de desarrollo

Etapa 1. Diseño. Definir la gramática reducida del pseudolenguaje, las instrucciones soportadas y el formato de salida.

Etapa 2. Expresiones. Implementar la conversión de expresiones infijas a prefijas y postfijas; comprobar la precedencia con al menos cinco expresiones.

Etapa 3. Código intermedio. Construir la generación de triplos y cuádruplos para asignaciones y operaciones.

Etapa 4. Control y funciones. Agregar una representación básica del condicional si y de llamadas a funciones.

Etapa 5. Integración. Unir los componentes, probar errores, corregir inconsistencias y preparar una demostración.

8. Pruebas mínimas que deben demostrar

• Una expresión con precedencia: x = 4 + 3 * 2;

• Una expresión con paréntesis: x = (4 + 3) * 2;

• Una asignación que use variables: total = precio * cantidad;

• Una comparación dentro de un si: si (total > 100) { ... }

• Una llamada a función: imprimir(total);

• Un caso con error y el mensaje que muestra el programa.

9. Entregables

• Código fuente en Python, ordenado y comentado en las partes importantes.

• Archivo de ejemplo con al menos cinco instrucciones de entrada.

• Documento breve con la gramática o reglas que admite el programa, instrucciones soportadas y limitaciones.

• Evidencia de pruebas: capturas de pantalla o salidas copiadas que muestren notaciones, triplos, cuádruplos y control.

• Exposición y demostración en la que cada integrante explique su contribución y el recorrido de una expresión.

10. Criterios de evaluación

Criterio	Valor	Qué se revisará
Notaciones	20 puntos	Prefija, infija y postfija correctas; precedencia y paréntesis.
Triplos y cuádruplos	25 puntos	Operaciones en orden correcto y referencias/temporales consistentes.
Variables, expresiones y asignación	15 puntos	Identificación de símbolos y traducción de asignaciones.
Control y funciones	15 puntos	Representación entendible de condicionales, saltos y llamadas.
Pruebas y manejo de errores	10 puntos	Casos variados y mensajes claros ante entradas incorrectas.
Organización y documentación	5 puntos	Código legible, comentarios útiles e instrucciones para ejecutarlo.
Exposición individual	10 puntos	Cada integrante explica y responde preguntas sobre el programa.

11. Preguntas para la presentación

• ¿Por qué la multiplicación se genera antes que la suma en a + b * 2?

• ¿Qué diferencia hay entre la notación postfija y los cuádruplos?

• ¿Cómo se representa el resultado de una operación en un triplo?

• ¿Qué información guarda la tabla de símbolos?

• ¿Cómo indica el código intermedio que una condición no se cumple?

• ¿Qué entradas no puede procesar todavía su programa y por qué?

12. Condiciones de aceptación

El proyecto se considera funcional si puede procesar expresiones y asignaciones del subconjunto definido, mostrar las tres notaciones para expresiones aritméticas, generar triplos y cuádruplos coherentes, representar al menos un condicional y una llamada a función, y explicar mediante un mensaje al menos un error de entrada. No se calificará por tener una interfaz sofisticada, sino por la corrección de la traducción y la comprensión del código generado.

Recomendación: construir primero una versión pequeña que traduzca expresiones y asignaciones. Después integrar triplos, cuádruplos, condicionales y funciones. Es preferible entregar un programa reducido que funcione correctamente a intentar soportar todo el lenguaje y no poder demostrarlo.